

Detailed Notes on the 3D INR Radar Case and the Synthetic-Data Validation Case

Working document

July 9, 2026

Contents

1	Purpose and scope of this document	4
2	The real 3D radar case	5
2.1	What the real case is trying to learn	5
2.2	Why an INR instead of an ordinary gridded interpolator	5
2.3	The model: a sine-activated MLP	5
2.4	The real-data dataset	6
2.5	The real-data training window	6
3	The loss function in the 3D case	7
3.1	The data loss	7
3.2	Curvature regularization	7
3.3	Reference-ratio weighting	8
3.4	Warmup	8
4	Collocation points, diagnostic points, and plotting grids	8
4.1	Why there are several different grids	8
4.2	What <code>collocation_grid_nx</code> and <code>collocation_grid_ny</code> mean	9
4.3	Why 80×80 was not absurd and why 100×100 is fine	9
4.4	Diagnostic collocation points	10
5	Derivative diagnostics	10
5.1	What is tracked	10
5.2	Why near-zero derivative fractions matter	11
5.3	Why component gradient norms matter	11
6	The synthetic-data case	12
6.1	Why the synthetic case was introduced	12
6.2	The first synthetic plasma field	12
6.3	Synthetic observations	13
6.4	The synthetic dataset class	13

7	The synthetic trainer	14
7.1	What the synthetic trainer should be	14
7.2	Why the trainer uses command-line arguments	14
7.3	Baseline synthetic training command	15
8	The current synthetic sweep	16
8.1	Why a sweep is needed	16
8.2	Data-only run	16
8.3	Baseline regularized run: $xy = 0.30, t = 0.30$	17
8.4	Stronger spatial regularization: $xy = 0.70, t = 0.30$	17
8.5	Stronger spatial and temporal regularization: $xy = 0.70, t = 0.70$	18
9	The synthetic analysis script	18
9.1	Why the analysis script uses top-of-file settings	18
9.2	Path anchoring issue and fix	19
9.3	What the analysis script computes	19
10	Interpretation of the first synthetic analysis plots	21
10.1	Scalar density reconstruction	21
10.2	Gradient reconstruction	21
10.3	Boundary and extrapolation effects	21
11	How to compare the sweep results	22
11.1	Run all four cases before deciding	22
11.2	Metrics to compare	22
11.3	Likely interpretations	23
12	Relationship between the synthetic case and the real radar case	23
12.1	What transfers directly	23
12.2	What does not transfer directly	24
12.3	Why the synthetic geometry should eventually become more realistic	24
13	Practical workflow for running the current experiments	25
13.1	Preparation	25
13.2	Run the four training cases	25
13.3	Analyze each run	25
13.4	Inspect training histories	25
13.5	Inspect error summaries	26
14	Why the analysis should be improved before over-interpreting results	26
14.1	Inner-domain mask	26
14.2	Near-sample mask	26
14.3	High-gradient mask	27
14.4	Observed-point metrics	27
15	What decisions have been made and why	27
15.1	Use $\log_{10} N_e$ instead of N_e	27
15.2	Use normalized coordinates and normalized targets	27
15.3	Use a sine-activated INR	27

15.4	Keep ω_0 moderate	27
15.5	Use curvature regularization instead of gradient regularization	27
15.6	Use reference-ratio regularization	28
15.7	Use command-line arguments for training	28
15.8	Use top-of-file settings for single-run analysis	28
15.9	Do a data-only control	28
15.10	Do stronger spatial and temporal regularization runs	28
16	Common failure modes and what they mean	28
16.1	Synthetic CSV not found	28
16.2	Diagnostics are NaN in data-only run	28
16.3	Scalar prediction looks good but gradient error is bad	28
16.4	Boundary artifacts dominate the error maps	29
16.5	Stronger regularization lowers derivative error but raises data loss	29
16.6	Stronger regularization smears the Gaussian	29
17	Near-term next steps	29
17.1	Finish the four-run regularization sweep	29
17.2	Run the analysis script on each run	29
17.3	Create a comparison script	29
17.4	Add regional metrics	30
17.5	Only then expand the synthetic physics	30
18	A note on automation	30
19	Final technical picture	31

1 Purpose and scope of this document

This document records, in detail, the workflow that has been built around the three-dimensional implicit neural representation (INR) radar reconstruction problem and the synthetic-data version of the same problem. The goal is not to provide a short recipe. The goal is to describe what the pieces are, why they were introduced, what decisions were made, what should remain identical between the real-data and synthetic-data cases, how the current runs are being launched, how the outputs should be analyzed, and what the next experiments mean physically and numerically.

The word “3D” in this project has a specific meaning. In the current implementation, the network input is

$$(x, y, t), \tag{1}$$

not yet a full spatial volume (x, y, z) plus time. The network is therefore learning a scalar field over two spatial coordinates and one time coordinate:

$$f_{\theta}(x, y, t) \approx \log_{10} N_e(x, y, t). \tag{2}$$

The model is continuous in its inputs, meaning that once it is trained it can be evaluated at arbitrary (x, y, t) coordinates within the normalized domain. This is the key reason an INR is useful here: the radar provides sparse samples, but the trained function can be evaluated on a dense plotting grid or on derivative-probe points.

There are two main branches of the work:

1. The real radar-data case, where samples come from AMISR/PFISR data loaded from an HDF5 file.
2. The synthetic-data case, where samples are generated from a known analytic plasma-density field, allowing us to compare the trained INR against the true field and its derivatives.

The synthetic case exists because the real radar case does not give us a dense ground truth field. With real radar data, we can evaluate the model at the measured points, inspect visual reconstructions, and track derivative diagnostics, but we cannot know the exact reconstruction error everywhere in the domain. The synthetic case solves that problem by creating fake radar observations from a known field. Then, after training on sparse fake observations, we can evaluate the trained network on a dense grid and compare against the exact synthetic truth.

The important design principle that emerged is this:

The synthetic-data trainer should be the same training pipeline as the real-data trainer, except for the source of the samples. The real-data dataset loader is replaced by a synthetic dataset loader. The loss, network, collocation logic, derivative diagnostics, output files, and analysis philosophy should otherwise remain the same.

That principle matters because the purpose of the synthetic case is to validate the same method that will be used on the radar case. If the synthetic trainer becomes a different simplified code path, then it stops being a faithful validation of the real workflow.

2 The real 3D radar case

2.1 What the real case is trying to learn

The real-data branch treats the radar observations as samples of a continuous plasma-density field. The target quantity is not N_e directly, but

$$f(x, y, t) = \log_{10} N_e(x, y, t). \quad (3)$$

This is a deliberate choice. Electron density can vary by orders of magnitude, and fitting N_e directly can create poor numerical conditioning. The logarithm compresses the dynamic range and makes the target smoother and more suitable for mean-squared-error training.

The model receives three coordinates:

$$\mathbf{r} = (x, y, t), \quad (4)$$

and returns one scalar:

$$\hat{f}_\theta(x, y, t). \quad (5)$$

Here, x and y are horizontal spatial coordinates in km, and t is time in seconds. In the code, these physical coordinates are normalized before entering the neural network. The target $\log_{10}(N_e)$ is also normalized during training. This is another important design decision: the network is trained in normalized coordinates and normalized target space, but analysis plots and CSV outputs should be interpreted in physical units after inverse transforms.

2.2 Why an INR instead of an ordinary gridded interpolator

The problem is not simply to draw a smooth picture through radar samples. The desired object is a differentiable function that can be evaluated continuously. This matters because the project is interested not only in density reconstructions but also in gradients and higher derivatives. A coordinate-based neural network gives us

$$\hat{f}_\theta(x, y, t), \quad (6)$$

and because it is built from differentiable operations, automatic differentiation can compute quantities such as

$$\frac{\partial \hat{f}}{\partial x}, \quad \frac{\partial \hat{f}}{\partial y}, \quad \frac{\partial \hat{f}}{\partial t}, \quad (7)$$

and also second derivatives such as

$$\frac{\partial^2 \hat{f}}{\partial x^2}, \quad \frac{\partial^2 \hat{f}}{\partial x \partial y}, \quad \frac{\partial^2 \hat{f}}{\partial y^2}, \quad \frac{\partial^2 \hat{f}}{\partial t^2}. \quad (8)$$

Those derivative quantities are central to the regularization and diagnostics described later.

2.3 The model: a sine-activated MLP

The current model is a multilayer perceptron with sine activations, similar in spirit to a SIREN. The basic object is a neural function

$$\hat{f}_\theta : \mathbb{R}^3 \rightarrow \mathbb{R}. \quad (9)$$

The input dimension is three, corresponding to (x, y, t) , and the output dimension is one, corresponding to normalized $\log_{10}(N_e)$.

Sine activations are useful in INR problems because they produce smooth, differentiable functions with nonzero higher derivatives. This is important when derivative-based losses or diagnostics are part of the method. A ReLU network, for example, is piecewise linear. Its second derivative is zero almost everywhere and undefined at activation boundaries. That makes it a poor choice when the training code explicitly computes second derivatives such as f_{xx} , f_{xy} , f_{yy} , and f_{tt} .

The frequency factor ω_0 controls the scale of variation represented by the sine activations. A larger ω_0 allows high-frequency oscillations more easily, but can also make training more sensitive and can produce ringing between sparse samples. A smaller ω_0 biases the network toward smoother functions. In earlier discussion, a value around $\omega_0 = 5$ was preferred for this radar problem rather than the more canonical SIREN value of 30, because the radar field is expected to be relatively smooth compared with image-like INR examples, and because the data sampling is sparse.

The point is not that $\omega_0 = 5$ is universally correct. The point is that it is a reasonable working value for the current reconstruction problem. It should be treated as one of the parameters that may later be swept, but it was not the first parameter chosen for the current synthetic comparison because the density reconstruction looked qualitatively good and the immediate question became the effect of curvature regularization.

2.4 The real-data dataset

The real-data dataset class is conceptually represented by `RadarTimeH5Dataset`. It reads AMIS-R/PFISR HDF5 data and creates training samples containing coordinates and targets. The important output fields are conceptually:

Quantity	Meaning
<code>x_km</code>	horizontal coordinate in km
<code>y_km</code>	horizontal coordinate in km
<code>t_sec</code>	time coordinate in seconds
<code>time_index</code>	discrete time-record index within the selected window
<code>log10_Ne</code>	target electron density in log base 10

The dataset also stores normalization/scaling information. This is essential because the training coordinate tensor is not in raw km and seconds. The network sees normalized coordinates. The physical coordinate ranges are still needed for plotting, dense evaluation, collocation construction, and analysis.

2.5 The real-data training window

The real-data training is performed over a window of time records. In the current discussion, a typical case uses around 31 time instances. With two-minute integration data, this corresponds to a limited time window rather than hours of data. That is important computationally: these are not multi-day training runs. A single training run can take minutes, which makes parameter sweeps realistic.

The training arguments select the start and size of the window:

```
--window_start_index 0
--window_size_records 31
```

The network therefore learns a continuous function over the selected spatial region and over the selected time span.

3 The loss function in the 3D case

3.1 The data loss

The most basic loss is the mean-squared error between the predicted and observed normalized target values at the radar sample points:

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^N \left(\hat{f}_\theta(x_i, y_i, t_i) - f_i \right)^2. \quad (10)$$

Here f_i is the observed normalized target corresponding to $\log_{10} N_e$ at the i -th radar sample.

If only this term is used, the network is asked only to fit the measured points. Since the data are sparse, there are infinitely many continuous functions that can fit the measured points while doing unreasonable things between them. This is why additional smoothness constraints were introduced.

3.2 Curvature regularization

The regularization used in this project is based on second derivatives of the network output. It does not penalize the first derivative directly. It penalizes curvature.

The total loss has the form

$$L_{\text{total}} = L_{\text{data}} + \lambda_{xy} L_{xy} + \lambda_t L_t. \quad (11)$$

The spatial curvature term is

$$L_{xy} = \left\langle f_{xx}^2 + 2f_{xy}^2 + f_{yy}^2 \right\rangle, \quad (12)$$

where the angle brackets indicate an average over collocation points, and

$$f_{xx} = \frac{\partial^2 f}{\partial x^2}, \quad f_{xy} = \frac{\partial^2 f}{\partial x \partial y}, \quad f_{yy} = \frac{\partial^2 f}{\partial y^2}. \quad (13)$$

The temporal curvature term is

$$L_t = \left\langle f_{tt}^2 \right\rangle, \quad (14)$$

with

$$f_{tt} = \frac{\partial^2 f}{\partial t^2}. \quad (15)$$

The interpretation is straightforward but important. The data loss says: fit the observations. The curvature regularization says: while fitting the observations, avoid unnecessary bending, ripples, and oscillations between the observations. A constant field has zero curvature. A linear gradient also has zero curvature. A smooth Gaussian has curvature, but if the regularization is not too strong it can still be represented. What is discouraged are high-frequency wiggles that are not required by the data.

This is why the regularization is relevant to sparse radar reconstruction. The radar samples constrain only part of the domain. Without some bias toward smoothness, the INR may fit the samples while inventing structure in unsampled regions. With too much smoothness regularization, however, the network may smear real structures. Therefore the regularization must be tested rather than assumed correct.

3.3 Reference-ratio weighting

A practical problem is choosing λ_{xy} and λ_t . Fixed numerical values are difficult to interpret because the raw magnitudes of the data loss and curvature losses depend on normalization, coordinate scales, network initialization, and training stage. The reference-ratio approach was introduced to make these weights more interpretable.

Instead of saying directly “set $\lambda_{xy} = 10^{-4}$,” the command can specify target ratios:

```
--reference_loss_weights
--target_xy_ratio 0.30
--target_t_ratio 0.30
```

The practical meaning is that the weighted curvature terms are scaled relative to a reference data-loss level. A target ratio of 0.30 means that the weighted regularization term is intended to be of order 30% of the reference data-loss scale, not necessarily exactly 30% of the instantaneous loss at every step.

This makes experiments easier to reason about. For example:

Setting	Meaning	Expected effect
xy=0.30, t=0.30	moderate spatial and temporal smoothing	baseline regularized run
xy=0.70, t=0.30	stronger spatial smoothing	may reduce spatial ringing
xy=0.70, t=0.70	stronger spatial and temporal smoothing	may reduce both spatial and temporal artifacts
no regularization	data loss only	useful control case

3.4 Warmup

The training command includes a warmup parameter:

```
--lambda_warmup_steps 500
```

The purpose is to avoid forcing the curvature penalty at full strength from the very beginning of training. Early in training, the network does not yet represent the data well. If regularization is too strong immediately, it can prevent the model from first finding the broad structure. Warmup allows the model to begin fitting the data and then gradually introduces the regularization.

A consequence is that very short smoke tests can be misleading. For example, if a run lasts only 500 steps and the warmup is also 500 steps, the regularization may never be fully active. This is why a 500-step run is useful only as a smoke test to confirm that the code runs, writes files, and produces non-crashing outputs. It should not be used to judge the scientific effect of the regularization.

4 Collocation points, diagnostic points, and plotting grids

4.1 Why there are several different grids

A major source of confusion was the difference between collocation grids and plotting grids. They are not the same thing.

There are at least three different sets of points:

1. **Radar or observation points:** points where the data loss is computed.

2. **Collocation points:** points where derivative-based regularization is computed during training.
3. **Plotting or dense evaluation points:** points where the trained model is evaluated after training to make images or compute error maps.

The plotting grid can be very fine because plotting is only a forward pass. If the grid is 250×250 , the trained network is evaluated at those points, but no second derivatives and no parameter-gradient backpropagation are required for the plot itself.

Collocation points are much more expensive. For each collocation point, the trainer may compute second derivatives such as f_{xx} , f_{xy} , f_{yy} , and f_{tt} . These require automatic differentiation graphs and are much more costly than ordinary forward evaluation. Therefore the collocation grid does not need to be as fine as the plotting grid, and the number of collocation points sampled per optimizer step is a major runtime parameter.

4.2 What `collocation_grid_nx` and `collocation_grid_ny` mean

The arguments

```
--collocation_grid_nx 100
--collocation_grid_ny 100
```

specify the size of the candidate spatial grid used to build the collocation pool. This is not the number of radar samples. It is not the plotting resolution. It is the spatial resolution of the candidate locations at which derivative regularization can be sampled.

For a 31-time-record window, a 100×100 spatial collocation grid corresponds to

$$100 \times 100 \times 31 = 310000 \tag{16}$$

candidate collocation points, assuming the collocation pool is built at each discrete time record. Then the argument

```
--num_collocation 16384
```

means that 16384 points are sampled from that larger pool per training step for the curvature loss.

This is why a 100×100 grid does not make 16384 collocation points useless. The candidate pool is much larger than 16384. The training step samples a subset. Different steps can sample different subsets. The number 16384 controls how many derivative-regularization points are used in one optimizer step; the grid size controls the spatial-temporal support from which those points are drawn.

4.3 Why 80×80 was not absurd and why 100×100 is fine

An 80×80 grid may sound coarse compared with a plotting grid, but if the domain is around 500 km wide, the spacing is roughly

$$\frac{500 \text{ km}}{79} \approx 6.3 \text{ km}. \tag{17}$$

That is already denser than the radar sample spacing in many sparse-beam situations. For a smooth Gaussian structure with a characteristic width of tens of km, this is not obviously too coarse.

However, 100×100 is a reasonable choice and is what was selected for the current sweep. It gives a somewhat denser candidate grid without making setup and memory unnecessarily large.

For sharper structures, cutouts, depletions, or narrow gradients, a larger grid such as 150×150 or 200×200 might later be needed. That is not the first thing being tested now.

4.4 Diagnostic collocation points

The argument

```
--num_diagnostic_collocation 16384
```

controls how many points are used for derivative diagnostics at summary intervals. This is separate from the collocation points used in the loss at every step.

The desired behavior is:

- `num_collocation`: affects training, because it changes how many derivative-regularization points are used in the loss.
- `num_diagnostic_collocation`: affects diagnostics and monitoring, because it changes how derivative statistics are measured and reported.

This separation is important for the data-only run. Even when regularization is turned off, we still want to compute derivative diagnostics. The training loss should be data-only, but the history file should still tell us whether the data-only network has large curvature, ringing, or near-zero derivative pathologies. If diagnostics become NaN merely because regularization is off, then the code is mixing two concepts that should be independent. Training regularization and diagnostic measurement should be decoupled.

5 Derivative diagnostics

5.1 What is tracked

The diagnostic trainer records not only the scalar data loss but also statistics of the derivative fields. The important quantities include

$$f_{xx}, \quad f_{xy}, \quad f_{yy}, \quad f_{tt}, \tag{18}$$

and derived diagnostic columns such as root-mean-square values, mean absolute values, maximum absolute values, and fractions near zero.

Representative history columns include:

```
fxx_rms  
fxy_rms  
fyy_rms  
ftt_rms  
fxx_meanabs  
fxy_meanabs  
fyy_meanabs  
ftt_meanabs  
fxx_maxabs  
fxy_maxabs  
fyy_maxabs  
ftt_maxabs  
fxx_frac_near_zero
```

```
fxi_frac_near_zero
fyy_frac_near_zero
ftt_frac_near_zero
fxx_frac_exact_zero
fxy_frac_exact_zero
fyy_frac_exact_zero
ftt_frac_exact_zero
grad_norm_total
grad_norm_data
grad_norm_xy_weighted
grad_norm_t_weighted
```

The derivative diagnostics are not the same as the regularization loss. The loss uses derivative quantities to guide training. The diagnostics measure derivative quantities to help us understand what the trained model is doing. In a good diagnostic trainer, the diagnostics should still be available even when the regularization terms are turned off.

5.2 Why near-zero derivative fractions matter

One reason to track near-zero and exact-zero fractions is to detect pathological derivative behavior. In a sine network, second derivatives should generally not be identically zero everywhere. If a large fraction of second derivatives are exactly zero, that can indicate an implementation problem, inappropriate activation choice, detached tensors, or diagnostic computation failure. For example, if a ReLU-like network were used, second derivatives would be zero almost everywhere, making curvature regularization ineffective. With sine activations, that should not happen unless something else is wrong.

5.3 Why component gradient norms matter

The component gradient norms estimate how much each loss component contributes to parameter updates. For example, the trainer can estimate the norm of the gradient from the data loss, from the weighted spatial regularization, and from the weighted temporal regularization. This helps answer questions like:

- Is the regularization numerically too weak to matter?
- Is the regularization dominating the data loss?
- Does the warmup behave as expected?
- Does increasing `target_xy_ratio` actually increase the spatial-regularization pressure?

The argument

```
--component_grad_every 500
```

sets how often these expensive component-gradient diagnostics are computed. In the current runs, it is set equal to `summary_every` so that the rows written to `history.csv` at summary steps also contain component-gradient values.

6 The synthetic-data case

6.1 Why the synthetic case was introduced

The real radar case does not provide a dense ground truth. This makes it hard to answer several important questions:

- Does the INR recover the correct structure between sparse radar samples?
- Does regularization improve the reconstruction or merely smooth away real features?
- Are the gradients produced by automatic differentiation physically meaningful?
- Are second derivatives usable, or are they dominated by neural artifacts?
- How do errors behave away from measured points and near domain boundaries?

The synthetic-data case was built to answer these questions under controlled conditions. We create a known plasma-density field, sample it sparsely as though it were radar data, train the same INR pipeline on those sparse samples, and then compare the trained network against the true synthetic field on a dense grid.

This is the right intermediate step before trusting derivative quantities on real radar data. If the INR cannot recover gradients and reasonable structure in a synthetic case where the truth is known, then it should not be trusted blindly on real radar data.

6.2 The first synthetic plasma field

The first synthetic field is deliberately simple: a background plasma density plus a moving Gaussian enhancement. The intended physical scale is:

- background density around 10^9 m^{-3} ,
- patch amplitude around 10^{12} m^{-3} ,
- a spatial domain around $500 \times 500 \text{ km}$,
- a time sequence with about 31 time records,
- sparse observations analogous to radar samples.

The density field is conceptually

$$N_e(x, y, t) = N_{e0} + A \exp \left[-\frac{(x - x_c(t))^2}{2\sigma_x^2} - \frac{(y - y_c(t))^2}{2\sigma_y^2} \right], \quad (19)$$

where N_{e0} is the background density, A is the patch amplitude, σ_x and σ_y set the patch width, and $(x_c(t), y_c(t))$ is the moving patch center.

The network target is

$$f(x, y, t) = \log_{10} [N_e(x, y, t)]. \quad (20)$$

There was an earlier issue where a stale or low-amplitude synthetic file had a maximum target near $\log_{10} N_e \approx 10.5$, which was not the intended high-amplitude test. The generator was updated so that the patch amplitude could be explicitly set with an argument such as

```
--patch_amplitude_m3 1e12
```

along with patch-width arguments such as

```
--patch_sigma_x_km ...
--patch_sigma_y_km ...
```

The high-amplitude case is the relevant one for the current synthetic experiments.

6.3 Synthetic observations

The synthetic generator writes an observation CSV, for example:

```
outputs/synthetic_high_amp_left_right/synthetic_observations.csv
```

The columns include the sparse observation coordinates, the true electron density, the target $\log_{10} N_e$, and analytic derivatives of the true synthetic field. Representative columns are:

```
time_index
t_sec
t_min
beam_id
x_km
y_km
Ne
log10_Ne_true
log10_Ne
true_dlog10Ne_dx_km
true_dlog10Ne_dy_km
true_dlog10Ne_dt_sec
sample_geometry
is_observed
```

The exact sparse geometry in the first synthetic test is not yet a perfect reproduction of a specific real AMISR beam pattern. It is a useful first test because it creates sparse samples in the same kind of (x, y, t) domain. Later, the synthetic generator can be upgraded to use actual azimuth/elevation pairs, real beam geometries, or physically motivated plasma velocities such as quiet-time drifts of order 400 m/s. That is future work. The first priority was to verify the INR method under a known field.

6.4 The synthetic dataset class

The synthetic dataset class was created to mimic the real radar dataset interface. This is a critical design decision. The synthetic trainer should not need a separate conceptual training system. It should receive the same kind of tensors and metadata that the real-data trainer expects.

Conceptually:

Real-data case	Synthetic-data case	Purpose
RadarTimeH5Dataset	SyntheticPlasmaTimeDataset	provide training samples
AMISR HDF5 file	synthetic observation CSV	data source
observed $\log_{10}(N_e)$	analytic synthetic $\log_{10}(N_e)$ sampled sparsely	target
no dense truth	known dense truth	validation

The most important point is that the training code should see a dataset with the same normalized coordinate and target behavior. That lets the synthetic case validate the real training code rather than validating a different simplified implementation.

7 The synthetic trainer

7.1 What the synthetic trainer should be

The synthetic trainer should be the diagnostic reference-ratio trainer with the data source changed. In the real case, the trainer reads an HDF5 radar dataset. In the synthetic case, it reads a synthetic CSV. Everything else should remain as similar as possible.

The current user-facing training script is referred to as

```
synthetic_train_3d.py
```

Conceptually it is the synthetic version of the diagnostic reference-regularized real-data trainer. Earlier file names included longer names such as `synthetic_train_3d_window_reference_reg_diagnostic.py`; the important thing is the behavior, not the filename. The current command being used calls `synthetic_train_3d.py`.

7.2 Why the trainer uses command-line arguments

The training script should be command-line driven because training is meant to be swept. A training run is not a one-off analysis. Many runs may be launched with different values of

- regularization ratios,
- number of training steps,
- number of collocation points,
- diagnostic collocation points,
- collocation grid resolution,
- learning rate,
- model size,
- random seed,
- time window.

This is why the training code should accept arguments. It can then be driven by bash scripts, shell loops, Codex, or manual commands without editing the Python file each time.

This is different from the analysis script, discussed later, where top-of-file configuration is preferred because one typically analyzes one completed run at a time.

7.3 Baseline synthetic training command

The current baseline run used the high-amplitude synthetic observation file, a 31-record window, reference-ratio regularization with $xy = 0.30$ and $t = 0.30$, 15000 training steps, doubled collocation points relative to the earlier smaller test, and a 100×100 collocation candidate grid.

The command is:

```
python3 synthetic_train_3d.py \  
--synthetic_csv outputs/synthetic_high_amp_left_right/synthetic_observations.csv \  
--output_dir outputs/synthetic_train_high_amp_win0_diag_1500 \  
--window_start_index 0 \  
--window_size_records 31 \  
--reference_loss_weights \  
--target_xy_ratio 0.30 \  
--target_t_ratio 0.30 \  
--epsilon_data 1e-6 \  
--lambda_warmup_steps 500 \  
--num_steps 15000 \  
--summary_every 500 \  
--component_grad_every 500 \  
--num_collocation 16384 \  
--num_diagnostic_collocation 16384 \  
--collocation_grid_nx 100 \  
--collocation_grid_ny 100
```

The output directory name in that command contains 1500, but the run uses `-num_steps 15000`. For future organization it is better for the folder name to contain 15000, because otherwise it becomes easy to confuse a 1500-step run and a 15000-step run.

A clearer version of the same baseline command would be:

```
python3 synthetic_train_3d.py \  
--synthetic_csv outputs/synthetic_high_amp_left_right/synthetic_observations.csv \  
--output_dir outputs/synthetic_train_high_amp_win0_xy030_t030_15000 \  
--window_start_index 0 \  
--window_size_records 31 \  
--reference_loss_weights \  
--target_xy_ratio 0.30 \  
--target_t_ratio 0.30 \  
--epsilon_data 1e-6 \  
--lambda_warmup_steps 500 \  
--num_steps 15000 \  
--summary_every 500 \  
--component_grad_every 500 \  
--num_collocation 16384 \  
--num_diagnostic_collocation 16384 \  
--collocation_grid_nx 100 \  
--collocation_grid_ny 100
```

8 The current synthetic sweep

8.1 Why a sweep is needed

The first synthetic analysis showed that the network reconstructs the scalar density field reasonably well, but the gradient fields show coherent artifacts such as banding, boundary effects, and ring-like residuals. This means the pipeline works, but derivative recovery still needs to be evaluated carefully.

The correct next step is not immediately to redesign the network. The correct next step is to compare controlled training conditions while changing only one conceptual family of parameters: the strength of regularization. The four-run sweep is meant to answer whether curvature regularization helps or hurts.

The four experiments are:

Run	Spatial regularization	Temporal regularization	Purpose
A	none	none	data-only control
B	0.30	0.30	current baseline
C	0.70	0.30	stronger spatial smoothing
D	0.70	0.70	stronger spatial and temporal smoothing

All other settings should remain the same: same synthetic CSV, same time window, same number of steps, same collocation settings, same model, same learning rate, same seed if the script exposes one, same diagnostics. Otherwise the comparison becomes ambiguous.

8.2 Data-only run

The data-only run removes the reference-ratio regularization flags. That means the loss is only

$$L_{\text{total}} = L_{\text{data}}. \quad (21)$$

It is not expected to produce the smoothest solution. It is a control run. If data-only gives lower measured-point loss but worse dense-domain or derivative errors, that tells us regularization is doing useful work. If data-only gives better dense truth error and better derivative error, then the current regularization may be too strong or incorrectly weighted.

The command is:

```
python3 synthetic_train_3d.py \  
  --synthetic_csv outputs/synthetic_high_amp_left_right/synthetic_observations.csv \  
  --output_dir outputs/synthetic_train_high_amp_win0_data_only_15000 \  
  --window_start_index 0 \  
  --window_size_records 31 \  
  --epsilon_data 1e-6 \  
  --lambda_warmup_steps 500 \  
  --num_steps 15000 \  
  --summary_every 500 \  
  --component_grad_every 500 \  
  --num_collocation 16384 \  
  --num_diagnostic_collocation 16384 \  
  --collocation_grid_nx 100 \  
  --collocation_grid_ny 100
```


After the data-only run finishes, it is important to check whether derivative diagnostic columns are non-NaN:

```
head -1 outputs/synthetic_train_high_amp_win0_data_only_15000/history.csv

tail outputs/synthetic_train_high_amp_win0_data_only_15000/history.csv
```

If the derivative diagnostics are missing or NaN, that does not necessarily invalidate the data-only training result, but it does mean the diagnostic code should be fixed so that diagnostics remain available even when regularization is off.

8.3 Baseline regularized run: $xy = 0.30$, $t = 0.30$

The baseline regularized run is:

```
python3 synthetic_train_3d.py \
--synthetic_csv outputs/synthetic_high_amp_left_right/synthetic_observations.csv \
--output_dir outputs/synthetic_train_high_amp_win0_xy030_t030_15000 \
--window_start_index 0 \
--window_size_records 31 \
--reference_loss_weights \
--target_xy_ratio 0.30 \
--target_t_ratio 0.30 \
--epsilon_data 1e-6 \
--lambda_warmup_steps 500 \
--num_steps 15000 \
--summary_every 500 \
--component_grad_every 500 \
--num_collocation 16384 \
--num_diagnostic_collocation 16384 \
--collocation_grid_nx 100 \
--collocation_grid_ny 100
```

This is the current reference point. It is neither the no-regularization case nor the strongest regularization case. It is the moderate smoothing case.

8.4 Stronger spatial regularization: $xy = 0.70$, $t = 0.30$

This run tests whether the visible spatial gradient artifacts can be reduced by increasing spatial curvature regularization while keeping temporal regularization unchanged.

```
python3 synthetic_train_3d.py \
--synthetic_csv outputs/synthetic_high_amp_left_right/synthetic_observations.csv \
--output_dir outputs/synthetic_train_high_amp_win0_xy070_t030_15000 \
--window_start_index 0 \
--window_size_records 31 \
--reference_loss_weights \
--target_xy_ratio 0.70 \
--target_t_ratio 0.30 \
--epsilon_data 1e-6 \
--lambda_warmup_steps 500 \
--num_steps 15000 \
--summary_every 500 \
--component_grad_every 500 \
--num_collocation 16384 \
```

```
--num_diagnostic_collocation 16384 \
--collocation_grid_nx 100 \
--collocation_grid_ny 100
```

If this run reduces gradient ringing and boundary artifacts without smearing the Gaussian patch too much, it suggests the baseline spatial regularization was too weak. If it smears the patch and increases density error, it may be too strong.

8.5 Stronger spatial and temporal regularization: $xy = 0.70$, $t = 0.70$

This run tests whether stronger temporal regularization helps the moving-patch evolution or instead oversmooths it.

```
python3 synthetic_train_3d.py \
--synthetic_csv outputs/synthetic_high_amp_left_right/synthetic_observations.csv \
--output_dir outputs/synthetic_train_high_amp_win0_xy070_t070_15000 \
--window_start_index 0 \
--window_size_records 31 \
--reference_loss_weights \
--target_xy_ratio 0.70 \
--target_t_ratio 0.70 \
--epsilon_data 1e-6 \
--lambda_warmup_steps 500 \
--num_steps 15000 \
--summary_every 500 \
--component_grad_every 500 \
--num_collocation 16384 \
--num_diagnostic_collocation 16384 \
--collocation_grid_nx 100 \
--collocation_grid_ny 100
```

This is not automatically better than the 0.70/0.30 case. If the synthetic patch motion is smooth, stronger temporal regularization may help. If the temporal evolution requires more flexibility than the penalty allows, it may increase error or smear the motion.

9 The synthetic analysis script

9.1 Why the analysis script uses top-of-file settings

The training script is command-line driven because many training runs will be launched in sweeps. The analysis script was designed differently. For analyzing a single completed run, the preference was to set paths and options clearly at the top of the script rather than pass many command-line arguments.

This is the intended style:

```
RUN_DIR = Path("outputs/synthetic_train_high_amp_win0_xy030_t030_15000")

SYNTHETIC_CSV = Path("outputs/synthetic_high_amp_left_right/synthetic_observations.csv"
)
SYNTHETIC_CONFIG = Path("outputs/synthetic_high_amp_left_right/synthetic_config.json")

CHECKPOINT_NAME = "model_final.pt"
ANALYSIS_SUBDIR = "error_analysis"
```

```
GRID_NX = 250
GRID_NY = 250

DOMAIN_MODE = "full_domain"
TIME_SELECTION = "first_middle_last"

COMPUTE_GRADIENT_ERRORS = True
SAVE_DENSE_CSV = True
```

Then the script is run simply as:

```
python3 synthetic_analyze_reconstruction.py
```

or, if the local filename has a misspelling or fixed-path variant, use the actual local filename. The important point is that the settings are at the top.

9.2 Path anchoring issue and fix

An error occurred when the analysis script was run from a parent directory rather than from inside the synthetic project folder. The script looked for

```
outputs/synthetic_high_amp_left_right/synthetic_observations.csv
```

relative to the current terminal directory, not relative to the script's directory. If the terminal was in

```
~/postdoc/inr-radar
```

rather than

```
~/postdoc/inr-radar/inf_fakedata_3d
```

the script searched the wrong place.

The correct fix is to anchor relative paths to the script location:

```
SCRIPT_DIR = Path(__file__).resolve().parent
```

and then resolve relative paths against `SCRIPT_DIR`. This makes the script robust to being called from either the project folder or a parent folder.

9.3 What the analysis script computes

The analysis script loads the trained model checkpoint, the synthetic observations, the synthetic configuration, and the dataset scalars. It evaluates the trained INR on a dense grid at selected times. The first version evaluates three times:

- first time record,
- middle time record,
- last time record.

For each selected time, it computes the true synthetic field and the model prediction on a dense grid. It then writes dense CSVs and produces figures comparing truth, prediction, and error.

The scalar-density quantities include:

$$f_{\text{true}} = \log_{10} N_{\text{true}}, \quad (22)$$

$$f_{\text{pred}} = \hat{f}_{\theta}, \quad (23)$$

$$e = f_{\text{pred}} - f_{\text{true}}. \quad (24)$$

The summary metrics include:

- RMSE,
- MAE,
- bias,
- maximum absolute error,
- 95th percentile absolute error.

When gradient errors are enabled, the script also computes model gradients using automatic differentiation and compares them with the analytic synthetic gradients:

$$\frac{\partial \hat{f}}{\partial x} \quad \text{vs.} \quad \frac{\partial f_{\text{true}}}{\partial x}, \quad (25)$$

$$\frac{\partial \hat{f}}{\partial y} \quad \text{vs.} \quad \frac{\partial f_{\text{true}}}{\partial y}, \quad (26)$$

$$\frac{\partial \hat{f}}{\partial t} \quad \text{vs.} \quad \frac{\partial f_{\text{true}}}{\partial t}. \quad (27)$$

The outputs are written under an **error_analysis** subdirectory, such as:

```
outputs/synthetic_train_high_amp_win0_xy030_t030_15000/error_analysis/
```

Representative outputs are:

```
error_summary_by_time.csv
error_summary_mean.csv
dense_reconstruction_time_0000.csv
dense_reconstruction_time_0015.csv
dense_reconstruction_time_0030.csv
truth_pred_error_time_0000.png
truth_pred_error_time_0015.png
truth_pred_error_time_0030.png
gradient_error_time_0000.png
gradient_error_time_0015.png
gradient_error_time_0030.png
```

10 Interpretation of the first synthetic analysis plots

10.1 Scalar density reconstruction

The first synthetic plots showed that the scalar density reconstruction is successful at a basic level. The network recovers the moving Gaussian patch, places it in roughly the right location, and reproduces the broad amplitude and shape. The truth and prediction panels are visually similar at the first, middle, and last selected times.

This is a good sign. It means the basic pipeline works:

synthetic generator $\rightarrow$ synthetic dataset $\rightarrow$ INR training $\rightarrow$ dense reconstruction $\rightarrow$ truth comparison. (28)

However, the error maps are structured. They are not purely random noise. The scalar error contains coherent lobes and boundary effects. That is not necessarily a failure, but it tells us that the model is filling unsampled regions in a smooth learned way rather than exactly recovering the analytic field everywhere.

10.2 Gradient reconstruction

The gradient reconstruction is qualitatively encouraging but not yet something to trust blindly. The network recovers the main gradient ring around the Gaussian patch. That means it learned not only the scalar bump but also some of the spatial variation.

At the same time, the predicted gradient fields show artifacts such as banding, boundary effects, and coherent residual structures. This is expected to some degree because derivatives amplify small scalar-field errors. A scalar reconstruction can look visually good while its gradient field still contains noticeable artifacts.

Therefore the current interpretation is:

- The density reconstruction is good enough to continue.
- The gradient reconstruction is qualitatively reasonable but not yet quantitatively validated.
- The curvature and second-derivative fields should not yet be trusted for scientific interpretation until more diagnostics and controlled comparisons are performed.

10.3 Boundary and extrapolation effects

A significant part of the visible error appears near domain boundaries or away from strongly constrained sample regions. This is expected. Sparse observations constrain the model unevenly. The network may be well constrained near the sample geometry and poorly constrained in regions far from observations. A full-domain error metric can therefore mix interpolation error and extrapolation/boundary error.

This motivates adding regional metrics. The next analysis improvement should separate:

1. full-domain error,
2. inner-domain error excluding boundary strips,
3. near-sample-region error,
4. high-gradient-region error around the Gaussian edge.

This is important because a model that looks imperfect over the full square domain may still be accurate in the region that is physically constrained by the radar geometry.

11 How to compare the sweep results

11.1 Run all four cases before deciding

The sweep should not be judged one run at a time. The purpose is comparative. The same analysis script should be run on each output directory, producing an `error_analysis` subdirectory for each. Then the metrics should be compared across runs.

The main questions are:

- Does the data-only run fit sparse observations well but produce worse dense-domain error?
- Does moderate regularization reduce dense error or derivative artifacts?
- Does stronger spatial regularization reduce gradient ringing?
- Does stronger temporal regularization improve temporal consistency or oversmooth the moving patch?
- Does any regularization setting reduce scalar density error while worsening gradient error, or vice versa?

11.2 Metrics to compare

For each run, compare scalar-density metrics:

- density RMSE,
- density MAE,
- density bias,
- density maximum absolute error,
- density 95th percentile absolute error.

For gradient validation, compare:

- $\partial f / \partial x$ RMSE,
- $\partial f / \partial y$ RMSE,
- $\partial f / \partial t$ RMSE,
- gradient magnitude error,
- 95th percentile gradient errors.

From `history.csv`, compare:

- final data loss,

- curvature loss terms,
- f_{xx} RMS,
- f_{xy} RMS,
- f_{yy} RMS,
- f_{tt} RMS,
- component gradient norms.

Visual inspection remains important. Error maps can reveal coherent artifacts that a single RMSE number hides. The best run should not be selected solely by lowest scalar RMSE if its gradients are worse or if it creates structured artifacts in physically important regions.

11.3 Likely interpretations

Several outcomes are possible.

Data-only is best in scalar error but worst in gradients. This would mean the network can fit the scalar samples but produces derivative artifacts when unconstrained. Regularization is then useful for derivative quality even if it slightly increases data loss.

Moderate regularization is best overall. This would justify keeping $xy = 0.30$, $t = 0.30$ as a default starting point.

Stronger spatial regularization improves gradients without hurting density. This would suggest $xy = 0.30$ was too weak and $xy = 0.70$ is a better default for smooth Gaussian-like structures.

Stronger spatial regularization smears the patch. This would indicate that the regularization is too strong relative to the data and should be reduced.

Stronger temporal regularization helps little. This would mean most artifacts are spatial rather than temporal for this synthetic case, and the temporal ratio can remain lower.

Stronger temporal regularization worsens the moving patch. This would mean the model needs temporal flexibility to represent the drift, and $t = 0.70$ is too high.

12 Relationship between the synthetic case and the real radar case

12.1 What transfers directly

The following components should transfer directly between synthetic and real cases:

- model architecture,
- coordinate normalization philosophy,

- log-density target,
- data loss,
- curvature regularization,
- reference-ratio weighting,
- collocation sampling,
- derivative diagnostics,
- training-window concept,
- history logging,
- plotting and reconstruction workflow.

This is why the synthetic dataset was built to mimic the real dataset. The synthetic case is useful only if it tests the same method.

12.2 What does not transfer directly

The synthetic case has known truth. The real radar case does not. This is the largest difference.

In the synthetic case, after training, we can compute:

$$\hat{f}(x, y, t) - f_{\text{true}}(x, y, t) \tag{29}$$

everywhere on a dense grid. We can also compute exact gradient errors.

In the real case, we can compute residuals at measured points, inspect reconstructed structures, and track derivative diagnostics, but we cannot compute a full-domain truth error. Therefore the synthetic case is where hyperparameters and confidence in derivatives should be calibrated. The real case is where those lessons are applied cautiously.

12.3 Why the synthetic geometry should eventually become more realistic

The first synthetic test is useful because it verifies the method under a known analytic field. But a later synthetic generator should include more realistic radar geometry and plasma dynamics. That may include:

- actual azimuth/elevation beam pairs,
- realistic sparse sampling patterns,
- quiet-time drift speeds around 400 m/s,
- different plasma structures such as enhancements, depletions, and multiple patches,
- realistic temporal evolution tied to flow velocities,
- possibly altitude-dependent slices later, if the project returns to a fuller spatial representation.

This should be done after the current Gaussian-patch validation is understood. Adding physical complexity before validating the basic pipeline would make failures harder to interpret.

13 Practical workflow for running the current experiments

13.1 Preparation

Work inside the synthetic project directory, for example:

```
cd ~/postdoc/inr-radar/inf_fakedata_3d
```

Confirm that the synthetic observation file exists:

```
ls outputs/synthetic_high_amp_left_right/synthetic_observations.csv
```

If paths are uncertain, search for synthetic observation files:

```
find outputs -name "synthetic_observations.csv" -print
```

13.2 Run the four training cases

Run the data-only case, the baseline regularized case, the stronger-spatial case, and the stronger-spatial-plus-temporal case. Keep all settings except the regularization setting identical.

The four commands are given in the current synthetic sweep section. The practical reference is the explicit command block for each run.

13.3 Analyze each run

For each completed run, edit the top of `synthetic_analyze_reconstruction.py` so that `RUN_DIR` points to that run's output directory. Then run:

```
python3 synthetic_analyze_reconstruction.py
```

Repeat for each run. Each run should then have its own `error_analysis` directory.

13.4 Inspect training histories

For a quick check of available columns:

```
head -1 outputs/<run_name>/history.csv | tr ',' '\n' | grep -E 'fxx|fxy|fyy|ftt|
grad_norm|diag_curv'
```

For a quick check of final values:

```
tail -n 5 outputs/<run_name>/history.csv
```

A more structured check can be done with Python:

```
python3 - <<'PY'
import pandas as pd
p = 'outputs/<run_name>/history.csv'
h = pd.read_csv(p)
cols = [
    'step', 'data_loss', 'curv_xy_raw', 'curv_t_raw',
    'fxx_rms', 'fxy_rms', 'fyy_rms', 'ftt_rms',
    'fxx_frac_near_zero', 'fxy_frac_near_zero',
    'fyy_frac_near_zero', 'ftt_frac_near_zero',
    'grad_norm_total', 'grad_norm_data',
    'grad_norm_xy_weighted', 'grad_norm_t_weighted'
```

```

]
cols = [c for c in cols if c in h.columns]
print(h[cols].tail())
PY

```

13.5 Inspect error summaries

After analysis, inspect:

```
cat outputs/<run_name>/error_analysis/error_summary_mean.csv
```

and:

```
cat outputs/<run_name>/error_analysis/error_summary_by_time.csv
```

The by-time summary is important because a run may perform well at the middle time but worse at the first or last time. Edge times can be harder because temporal interpolation/extrapolation is less constrained there.

14 Why the analysis should be improved before over-interpreting results

The current analysis compares full-domain dense truth and prediction. That is useful, but not sufficient. The full square domain includes regions that may be poorly constrained by sparse observations. A single full-domain RMSE can therefore overemphasize boundary or extrapolation regions.

The next analysis improvement should add regional masks:

14.1 Inner-domain mask

Exclude a boundary strip, for example 5%, 10%, or a fixed number of km from each edge. This checks whether large errors are mostly boundary effects.

14.2 Near-sample mask

Compute distance from each dense grid point to the nearest observation point at the same time. Then define masks such as:

$$d_{\text{nearest}} < 25 \text{ km}, \quad (30)$$

or

$$d_{\text{nearest}} < 50 \text{ km}. \quad (31)$$

This separates interpolation near observed locations from extrapolation far away.

14.3 High-gradient mask

Use the true synthetic gradient magnitude to identify the physically important patch edge:

$$|\nabla f_{\text{true}}| = \sqrt{f_x^2 + f_y^2}. \quad (32)$$

Then compute metrics only where this value is large, for example above the 75th or 90th percentile. This tells us whether the network recovers the most important gradient region, not just the easy flat background.

14.4 Observed-point metrics

Also compute metrics at the sparse observation points. This is not enough by itself, because the model is trained on those points, but it is still useful to compare observed-point fit with dense-domain generalization.

15 What decisions have been made and why

15.1 Use $\log_{10} N_e$ instead of N_e

This was chosen because electron density has a large dynamic range. The logarithm makes the target scale easier to learn and makes errors more interpretable in terms of orders of magnitude.

15.2 Use normalized coordinates and normalized targets

This improves numerical conditioning. The physical units are km and seconds, which have different scales. Feeding raw coordinates directly can bias optimization. Normalization makes the neural network see inputs of comparable scale.

15.3 Use a sine-activated INR

Sine activations support smooth continuous fields and meaningful derivatives. This matters because the method computes first and second derivatives of the network output. A piecewise-linear activation would not be appropriate for curvature regularization.

15.4 Keep ω_0 moderate

A moderate value such as $\omega_0 = 5$ biases the model toward smoother behavior. Since the radar data are sparse and the structures being tested are broad, this is reasonable. Higher values can be tested later, but the immediate comparison is regularization strength, not frequency scaling.

15.5 Use curvature regularization instead of gradient regularization

The goal is not to forbid gradients. Plasma density can have real gradients. The goal is to discourage unnecessary bending and oscillation. Penalizing curvature allows linear gradients and smooth large-scale changes while suppressing high-frequency wiggles.

15.6 Use reference-ratio regularization

Reference-ratio weighting makes the regularization strength more interpretable than raw lambda values. It allows commands such as `target_xy_ratio 0.30` and `target_xy_ratio 0.70`, which are easier to compare.

15.7 Use command-line arguments for training

Training is meant to be run many times. Command-line arguments make sweeps possible without editing Python source files.

15.8 Use top-of-file settings for single-run analysis

Analysis is usually applied to one completed run at a time. Top-of-file configuration is clearer and less error-prone than passing many arguments, especially while the analysis script is still evolving.

15.9 Do a data-only control

The data-only run is necessary because otherwise we cannot tell whether regularization helps. It is possible for regularization to improve derivative behavior, but it is also possible for it to oversmooth the field. The no-regularization run is the control case.

15.10 Do stronger spatial and temporal regularization runs

The first synthetic result suggested that scalar density is reconstructed well but gradients have artifacts. Stronger spatial regularization may reduce spatial ringing. Stronger temporal regularization may or may not help motion consistency. Both must be tested rather than assumed.

16 Common failure modes and what they mean

16.1 Synthetic CSV not found

If the analysis script says:

```
FileNotFoundError: Synthetic CSV not found: outputs/...
```

then either the path is wrong or the script is resolving relative paths from the wrong current directory. The robust fix is to anchor paths to `Path(file.resolve().parent)`.

16.2 Diagnostics are NaN in data-only run

If the data-only run has NaN derivative diagnostics, it probably means the code computes diagnostics only when regularization is active. That should be fixed. The training loss and diagnostic measurement are different concepts.

16.3 Scalar prediction looks good but gradient error is bad

This is not surprising. Differentiation amplifies small scalar errors. A visually good scalar field can still have derivative artifacts. This is why synthetic validation is needed.

16.4 Boundary artifacts dominate the error maps

This means full-domain metrics may be mixing constrained and unconstrained regions. Add regional metrics before concluding that the method fails.

16.5 Stronger regularization lowers derivative error but raises data loss

This may be acceptable depending on the goal. If the purpose is derivative recovery and physically smooth interpolation, a slightly higher data loss may be worth it. If the purpose is exact fit to observed points, it may not be.

16.6 Stronger regularization smears the Gaussian

Then the regularization is too strong for that structure, or the model/warmup/weighting needs adjustment. Try a lower ratio or different warmup before changing the architecture.

17 Near-term next steps

The next steps should remain controlled.

17.1 Finish the four-run regularization sweep

Run:

1. data-only,
2. $xy = 0.30$, $t = 0.30$,
3. $xy = 0.70$, $t = 0.30$,
4. $xy = 0.70$, $t = 0.70$.

Do not change other parameters during this sweep.

17.2 Run the analysis script on each run

Each output directory should have an `error_analysis` directory with scalar and gradient comparison files.

17.3 Create a comparison script

The next useful code addition is a script that reads all four `error_analysis` directories and produces a combined table and plots. It should not modify the trainer. It should only compare completed runs.

The script should collect:

- final history metrics,
- mean scalar error metrics,
- by-time scalar error metrics,

- mean gradient error metrics,
- derivative diagnostic summaries.

It should produce:

- a combined CSV table,
- density error comparison plots,
- gradient error comparison plots,
- loss-history comparison plots,
- derivative-diagnostic comparison plots.

17.4 Add regional metrics

Before using the synthetic result to set final hyperparameters, add inner-domain, near-sample, and high-gradient masks.

17.5 Only then expand the synthetic physics

After the current Gaussian-patch test is understood, the synthetic generator can be made more realistic with actual beam azimuth/elevation pairs, quiet-time drift speeds, and more physical structures. That should be a second stage, not mixed into the first validation sweep.

18 A note on automation

The useful automation here is not to let an agent randomly change the research code. The useful automation is to make repeated short experiments reproducible. Since a single 31-record run can take only a few minutes, automation can run controlled sweeps, analyze outputs, and compare metrics.

The safest automated workflow is:

1. preserve the original working repository,
2. run experiments in a clearly separate working copy or branch,
3. make training scripts write logs and outputs to explicit directories,
4. make analysis scripts read completed outputs and produce comparison tables,
5. avoid modifying the trainer unless the requested experiment requires it.

The scientific rule is the same whether work is done manually or by an agent: change one thing at a time and make the output directories encode the changed parameter.

19 Final technical picture

The project is now at a good stage. The real 3D INR pipeline exists and uses a differentiable neural field $f_\theta(x, y, t)$ to reconstruct $\log_{10} N_e$ from sparse radar samples. The synthetic version now allows controlled validation by replacing real radar samples with sparse samples from a known analytic plasma field. The first synthetic reconstruction showed that the density field can be recovered reasonably well, but derivative fields still show artifacts that need quantitative comparison.

The current work is therefore not about proving that one image looks good. It is about determining whether the same INR pipeline can produce reliable scalar reconstructions and derivative estimates under controlled conditions. The four-run sweep is the correct next experiment because it directly tests the role of curvature regularization:

$$L_{\text{total}} = L_{\text{data}} + \lambda_{xy} L_{xy} + \lambda_t L_t. \quad (33)$$

The data-only run tells us what happens without smoothness pressure. The 0.30/0.30 run is the current moderate baseline. The 0.70/0.30 run tests stronger spatial smoothing. The 0.70/0.70 run tests stronger spatial and temporal smoothing. The correct choice among them should be based on scalar-density error, gradient error, derivative diagnostics, and visual inspection of structured artifacts.

Once that is understood, the next scientific step is to make the synthetic geometry and plasma dynamics more realistic. That is where actual beam azimuth/elevation information and physically motivated velocities can enter. But the present Gaussian-patch case is exactly the right first validation case because the truth is known, the interpretation is clean, and failures are easier to diagnose.